SIMATS ENGINEERING

ASSESSMENT TOOL 1: Knowledge Representation Design Assessment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA0101**

COURSE OUTCOME COVERED

CO3: *Implement propositional and first-order logic using resolution, unification, and inference techniques for knowledge representation and reasoning.*(BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100 **Time: 60 Mins**

No. of Questions: 5

Annexure A

Knowledge Representation Design Assessment

Code of Conduct:

I **M. Veera Naga Sai (192425253)(***Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: M Veera Naga Sai

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Engineering and Knowledge Base Design	15%	
3. Propositional Logic Representation	10%	
4. First-Order Logic Representation	15%	
5. Unification and Resolution	15%	
6. Forward and Backward Chaining	15%	
7. Inference and Reasoning Accuracy	10%	
8. System Architecture / Representation Diagram	5%	
9. Prototype Implementation and Results	5%	
Total	100%	

Signature of the Faculty Total Marks Awarded

ANSWER ANY ONE QUESTION

1. Smart Healthcare Diagnosis System

A hospital wants to develop an AI-based system that can identify possible illnesses from patient symptoms. The system contains facts such as:

- Ravi has fever.
- Ravi has cough.
- Ravi has body pain.
- Patients with fever and cough may have flu.
- Patients with flu and body pain may have viral infection.
- Patients with viral infection should be advised to rest.

Task:

1. Represent the given knowledge using **Propositional Logic**.
2. Convert the knowledge into **First-Order Logic (FOL)**.
3. Construct a knowledge base containing facts and rules.
4. Demonstrate **unification** for at least two predicates.
5. Use **forward chaining** to determine Ravi's condition.
6. Use **backward chaining** to prove whether Ravi needs rest.
7. Demonstrate **resolution** for one selected query.
8. Explain the final inference produced by the system.

2. Intelligent Loan Approval System

A bank wants to develop an AI system to determine whether a customer is eligible for a loan. The knowledge base contains information about income, credit score, employment, existing loans, and repayment history.

For example:

- Arun has a stable job.
- Arun has a high credit score.
- Arun has sufficient income.
- Customers with stable employment and sufficient income are eligible for preliminary approval.
- Customers with high credit scores and good repayment history are considered low-risk.
- Low-risk customers with preliminary approval can be recommended for loan approval.

Task:

1. Identify the **entities, facts, predicates, and rules** involved.
2. Represent the knowledge using **First-Order Logic**.
3. Build a knowledge base with at least **10 facts and 8 rules**.
4. Demonstrate **unification and substitution**.

5. Apply **forward chaining** to determine whether Arun can be recommended for approval.
6. Apply **backward chaining** for the query `LoanApproved(Arun)`.
7. Use **resolution** to prove or disprove one query.
8. Discuss how the knowledge-engineering process helps maintain the loan rules.

3. Intelligent Vehicle Fault Diagnosis

An automobile service center wants to develop an expert system that identifies possible vehicle faults based on symptoms.

The system knows that:

- A vehicle has a dead battery if it does not start and the headlights are dim.
- A vehicle may have a fuel problem if the engine cranks but does not start.
- A vehicle may have an overheating problem if the temperature is high and coolant is low.
- A vehicle should be sent for immediate service if overheating is detected.

For a vehicle named **Car101**, the system receives:

- Car101 does not start.
- Car101 has dim headlights.
- Car101 has a high temperature.

Task:

1. Design the **knowledge representation** for the vehicle diagnosis system.
2. Represent at least five rules using **FOL**.
3. Represent selected statements using **Propositional Logic**.
4. Demonstrate **unification** using suitable predicates.
5. Apply **forward chaining** to identify possible faults.
6. Apply **backward chaining** for the query `NeedsService(Car101)`.
7. Demonstrate **resolution** for one diagnosis.
8. Explain how inference is used to arrive at the final diagnosis.

4. Intelligent Student Academic Advising System

A university wants an AI-based academic advising system that recommends actions to students based on their academic performance.

The system contains rules such as:

- Students with attendance below 75% require attendance counseling.
- Students who fail two or more courses require academic counseling.
- Students with high attendance and good academic performance can be recommended for advanced courses.
- Students who have completed prerequisite courses can register for the corresponding advanced course.

Consider a student **Priya** who has:

- Attendance of 68%.
- Two failed courses.
- Completed the prerequisite for an advanced AI course.

Task:

1. Identify the **knowledge engineering requirements** for the system.
2. Represent the academic rules using **FOL**.
3. Create a knowledge base containing facts and rules.
4. Demonstrate **unification** using student and course predicates.
5. Use **forward chaining** to identify recommendations for Priya.
6. Use **backward chaining** to determine whether Priya requires academic counseling.
7. Apply **resolution** to prove one selected recommendation.
8. Explain the advantages and limitations of using a rule-based inference system for academic advising.

5. Cybersecurity Threat Detection System

An organization wants to develop an AI knowledge-based system to identify suspicious activities in its network.

The knowledge base contains rules such as:

- Multiple failed login attempts from the same user may indicate a brute-force attack.
- A login from an unusual location combined with multiple failed attempts indicates suspicious activity.
- Suspicious activity followed by unauthorized file access indicates a possible security breach.
- A possible security breach should generate a high-priority alert.

For user **User101**, the system observes:

- Five failed login attempts.
- Login from an unusual location.
- Unauthorized access to a confidential file.

Task:

1. Design a **knowledge representation model** for the cybersecurity domain.
2. Represent the knowledge using **Propositional Logic and FOL**.
3. Construct at least **10 facts and 8 rules**.
4. Demonstrate **unification and substitution**.
5. Apply **forward chaining** to determine whether a security alert should be generated.
6. Apply **backward chaining** for the query `HighPriorityAlert(User101)`.
7. Use **resolution** to prove the existence of a possible security breach.
8. Compare **forward chaining and backward chaining** for cybersecurity reasoning and justify which is more suitable.

ANSWER:

Smart Healthcare Diagnosis System

1.Representation Using Propositional Logic

Propositional Logic represents knowledge using simple statements that are either true or false.

Let:

- F = Ravi has fever
- C = Ravi has cough
- B = Ravi has body pain
- L = Ravi has flu
- V = Ravi has viral infection
- R = Ravi should be advised to rest

The given facts are:

F

C

B

The rules are:

1. If a patient has fever and cough, then the patient may have flu.

$(F \land C) \rightarrow L$

2. If a patient has flu and body pain, then the patient may have viral infection.

$(L \land B) \rightarrow V$

3. If a patient has viral infection, then the patient should be advised to rest.

$V \rightarrow R$

Therefore, the complete propositional knowledge base is:

$KB = \{F, C, B, (F \land C) \rightarrow L, (L \land B) \rightarrow V, V \rightarrow R\}$

2. Conversion into First-Order Logic (FOL)

First-Order Logic represents objects, their properties, and relationships using predicates and variables.

Let:

Fever(x) = x has fever

Cough(x) = x has cough

BodyPain(x) = x has body pain

Flu(x) = x has flu

ViralInfection(x) = x has viral infection

Rest(x) = x should be advised to rest

Here, Ravi is a constant representing the patient.

Facts:

1. Fever(Ravi)
2. Cough(Ravi)
3. BodyPain(Ravi)

Rules:

1. Patients with fever and cough may have flu:

$\forall x\ [(Fever(x) \land Cough(x)) \rightarrow Flu(x)]$

2. Patients with flu and body pain may have viral infection:

$\forall x\ [(Flu(x) \land BodyPain(x)) \rightarrow ViralInfection(x)]$

3. Patients with viral infection should be advised to rest:

$\forall x\ [ViralInfection(x) \rightarrow Rest(x)]$

Thus, the FOL knowledge base represents both the specific facts about Ravi and the general medical rules.

3. Construction of Knowledge Base

A Knowledge Base (KB) consists of facts and rules used by the AI system for reasoning.

Facts:

- Fever(Ravi)
- Cough(Ravi)
- BodyPain(Ravi)

Rules:

- $Fever(x) \land Cough(x) \rightarrow Flu(x)$
- $Flu(x) \land BodyPain(x) \rightarrow ViralInfection(x)$
- $ViralInfection(x) \rightarrow Rest(x)$

Therefore:

KB = {Fever(Ravi), Cough(Ravi), BodyPain(Ravi), Fever(x) ∧ Cough(x) → Flu(x), Flu(x) ∧ BodyPain(x) → ViralInfection(x), ViralInfection(x) → Rest(x)}
The knowledge base can use these facts and rules to derive new information about the patient's condition.

4. Demonstration of Unification

Unification is the process of finding substitutions for variables so that two predicates become identical.

Example 1: Unification of Fever

Consider:

Fever(x)

Fever(Ravi)

The substitution is:

x = Ravi

After substitution:

Fever(Ravi) = Fever(Ravi)

Therefore, the two predicates are successfully unified.

Most General Unifier (MGU):

{x/Ravi}

Example 2: Unification of Cough

Consider:

Cough(x)

Cough(Ravi)

The substitution is:

x = Ravi

After substitution:

Cough(Ravi) = Cough(Ravi)

Therefore, these predicates are also successfully unified.

Most General Unifier (MGU):

{x/Ravi}

Thus, unification allows the general rules containing variables to be applied to specific patients such as Ravi.

5. Forward Chaining to Determine Ravi's Condition

Forward chaining is a data-driven reasoning method. It starts with the known facts and applies rules to generate new facts.

Step 1: Initial Facts

The system knows:

Fever(Ravi)

Cough(Ravi)

BodyPain(Ravi)

Step 2: Apply Rule 1

Rule:

$Fever(x) \land Cough(x) \rightarrow Flu(x)$

Since Ravi has both fever and cough:

$Fever(Ravi) \land Cough(Ravi)$

Therefore:

Flu(Ravi)

Step 3: Apply Rule 2

Rule:

$Flu(x) \land BodyPain(x) \rightarrow ViralInfection(x)$

The system now knows:

Flu(Ravi)

BodyPain(Ravi)

Therefore:

ViralInfection(Ravi)

Step 4: Apply Rule 3

Rule:

$ViralInfection(x) \rightarrow Rest(x)$

Since:

ViralInfection(Ravi)

Therefore:

Rest(Ravi)

Forward Chaining Result

Fever(Ravi) + Cough(Ravi)

→ Flu(Ravi)

→ ViralInfection(Ravi)

→ Rest(Ravi)

Therefore, forward chaining concludes that Ravi may have a viral infection and should be

advised to rest.

6. Backward Chaining to Prove Whether Ravi Needs Rest

Backward chaining is a goal-driven reasoning method. It starts with the desired conclusion and works backward to find supporting facts.

Goal:

Rest(Ravi)

To prove Rest(Ravi), the system searches for a rule that produces Rest.

Rule:

ViralInfection(x) → Rest(x)

Therefore, the system needs to prove:

ViralInfection(Ravi)

To prove ViralInfection(Ravi), use:

Flu(x) ∧ BodyPain(x) → ViralInfection(x)

Therefore, the system needs:

Flu(Ravi)

BodyPain(Ravi)

BodyPain(Ravi) is already a fact.

Now, to prove Flu(Ravi), use:

Fever(x) ∧ Cough(x) → Flu(x)

The system already has:

Fever(Ravi)

Cough(Ravi)

Therefore:

Flu(Ravi)

Then:

Flu(Ravi) ∧ BodyPain(Ravi)

→ ViralInfection(Ravi)

Finally:

ViralInfection(Ravi)

→ Rest(Ravi)

Backward Chaining Result

Rest(Ravi)

← ViralInfection(Ravi)

← Flu(Ravi) + BodyPain(Ravi)

← Fever(Ravi) + Cough(Ravi) + BodyPain(Ravi)

All the required facts are available in the knowledge base.

Therefore, the system successfully proves that Ravi should be advised to rest.

7. Resolution for a Selected Query

Let the selected query be:

"Does Ravi have a viral infection?"

Query:

ViralInfection(Ravi)

For resolution, the negation of the query is added:

¬ViralInfection(Ravi)

The rules are converted into clause form.

Clause 1:

Fever(x) ∧ Cough(x) → Flu(x)

becomes:

¬Fever(x) ∨ ¬Cough(x) ∨ Flu(x)

Clause 2:

Flu(x) ∧ BodyPain(x) → ViralInfection(x)

becomes:

¬Flu(x) ∨ ¬BodyPain(x) ∨ ViralInfection(x)

Given Facts:

Fever(Ravi)

Cough(Ravi)

BodyPain(Ravi)

Resolution Steps:

From:

Fever(Ravi)

Cough(Ravi)

¬Fever(Ravi) ∨ ¬Cough(Ravi) ∨ Flu(Ravi)

we derive:

Flu(Ravi)

Next, using:

Flu(Ravi)

BodyPain(Ravi)

¬Flu(Ravi) ∨ ¬BodyPain(Ravi) ∨ ViralInfection(Ravi)

we derive:

ViralInfection(Ravi)

The negated query is:

¬ViralInfection(Ravi)

Therefore, we have:

ViralInfection(Ravi)

and

¬ViralInfection(Ravi)

Resolving these two statements produces the empty clause:

□

The empty clause indicates a contradiction. Therefore, the negated query is false.

Hence:

ViralInfection(Ravi)

is proven to be true according to the knowledge base.

8. Final Inference Produced by the System

The complete inference process is:

Ravi has fever

+

Ravi has cough

↓

Ravi may have flu

↓

Ravi has body pain

↓

Ravi may have viral infection

↓

Ravi should be advised to rest

Therefore, the final inference produced by the knowledge representation system is:

Ravi may have a viral infection, and the system advises Ravi to take rest.

The system reaches this conclusion using the facts and rules stored in the knowledge base. Forward chaining derives the conclusion from the available facts, while backward chaining starts with the goal of proving that Ravi needs rest. Resolution also confirms the conclusion through contradiction.

Note: The conclusion is based only on the given knowledge base and represents a logical inference, not an actual medical diagnosis.

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
Problem Analysis and Understanding	15%	13–15% – Clearly understands and analyzes the real-world problem, objectives, entities, facts, constraints, and expected inference.	10–12% – Understands the problem with minor omissions.	7–9% – Shows partial understanding with some important elements missing.	0–6% – Poor or incorrect understanding of the problem.
Knowledge Engineering & Knowledge Base Design	15%	13–15% – Develops a complete, consistent, and well-structured knowledge base with appropriate facts, predicates, entities, and rules.	10–12% – Knowledge base is well designed with minor inconsistencies or omissions.	7–9% – Basic knowledge base is developed but several facts/rules are missing or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
Propositional & First-Order Logic Representation	20%	17–20% – Correctly represents the domain using propositions, predicates, variables, constants, quantifiers, and logical connectives with excellent accuracy.	13–16% – Logic representation is mostly correct with minor errors.	9–12% – Provides basic representations but has several logical or syntactic errors.	0–8% – Logic representation is incorrect, incomplete, or missing.
Unification & Resolution	15%	13–15% – Correctly performs substitutions, identifies the MGU, and applies resolution systematically to derive valid conclusions.	10–12% – Applies unification and resolution correctly with minor errors.	7–9% – Demonstrates partial understanding but misses important steps or contains errors.	0–6% – Unable to correctly apply unification or resolution.

Forward Chaining & Backward Chaining	15%	13–15% – Correctly applies both forward and backward chaining with clear step-by-step reasoning and appropriate rule selection.	10–12% – Correctly applies both methods with minor errors or omissions.	7–9% – Demonstrates partial understanding of one or both chaining methods.	0–6% – Incorrectly applies or fails to demonstrate chaining techniques.
Inference & Reasoning Accuracy	10%	9–10% – Produces logically valid conclusions and clearly explains how facts and rules lead to the final inference.	7–8% – Conclusions are mostly correct with minor reasoning gaps.	5–6% – Some correct inferences but reasoning is incomplete or unclear.	0–4% – Conclusions are incorrect, unsupported, or missing.
Knowledge Representation Diagram / System Architecture	5%	5% – Provides a complete, accurate, well-labelled diagram showing knowledge base, inference engine, reasoning process, and output.	4% – Diagram is mostly correct with minor omissions.	3% – Diagram is partially complete or lacks clarity.	0–2% – Diagram is missing or incorrect.
Originality, Critical Thinking & Presentation	5%	5% – Demonstrates excellent independent thinking, appropriate real-world application, comparison of reasoning approaches, and clear presentation.	4% – Demonstrates good reasoning and clear presentation.	3% – Shows limited critical thinking and basic presentation.	0–2% – Shows little originality, weak reasoning, or poor presentation.